

```
import matplotlib.pyplot as plt
import numpy as np
from skimage.io import io
from skimage.transform import imread, imresize, downscale_local_mean
from skimage import img_as_uint
from skimage.color import rgb2gray
from skimage.color import gray2rgb
from skimage.color import rgb2hsv
from skimage.color import hsv2rgb
import cv2
from skimage.transform import resize
from skimage.util import img_as_ubyte
```

Analog Image

```
import matplotlib.pyplot as plt
from matplotlib.image import imread

def show_analog_image(Analog):
    # load image
    image = imread(Analog)

    # display the image
    plt.figure(figsize=(15,10))
    plt.imshow(image)
    plt.title('Example of an analog image')
    plt.axis('off')
    plt.show()

# display analog image
show_analog_image('Analog.png')
```



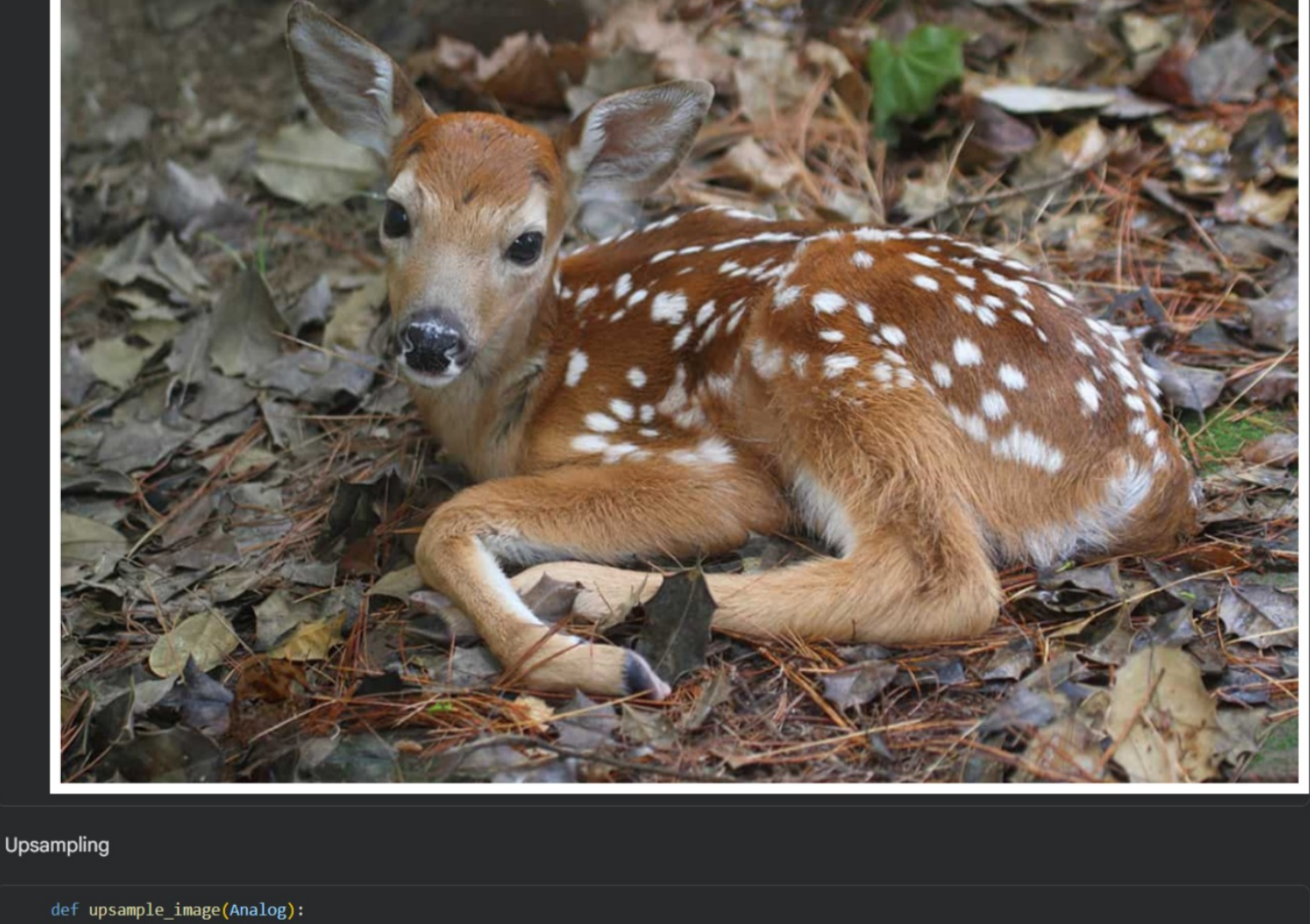
Digital Image

```
import matplotlib.pyplot as plt
from matplotlib.image import imread

def show_analog_image(Digital):
    # load image
    image = imread(Digital)

    # display the image
    plt.figure(figsize=(15,10))
    plt.imshow(image)
    plt.title('Example of an digital image')
    plt.axis('off')
    plt.show()

# display analog image
show_analog_image('Digital.jpg')
```



Upsampling

```
def upsample_image(Analog):
    # load the original image
    image = imread(Analog)

    # create upscaled images manually
    # resize image

    # scale 1 -> original size(no enlargement)
    image_1 = resize(
        image,
        (image.shape[0]*1, image.shape[1]*1),
        anti_aliasing=True
    )

    # scale 2 -> image become 2* larger
    image_2 = resize(
        image,
        (image.shape[0]*2, image.shape[1]*2),
        anti_aliasing=True
    )

    # scale 4 -> image become 4* larger
    image_4 = resize(
        image,
        (image.shape[0]*4, image.shape[1]*4),
        anti_aliasing=True
    )

    # scale 8 -> image become 8* larger
    image_8 = resize(
        image,
        (image.shape[0]*8, image.shape[1]*8),
        anti_aliasing=True
    )

    # create plotting area (1 row and 5 columns)
    fig, ax = plt.subplots(1, 5, figsize=(20,10))

    #display image
    #original image
    ax[0].imshow(image)
    ax[0].set_title('Original N=%d' % image.shape[0])

    #scale 01
    ax[1].imshow(image_1)
    ax[1].set_title('Scale 1 N=%d' % image_1.shape[0])

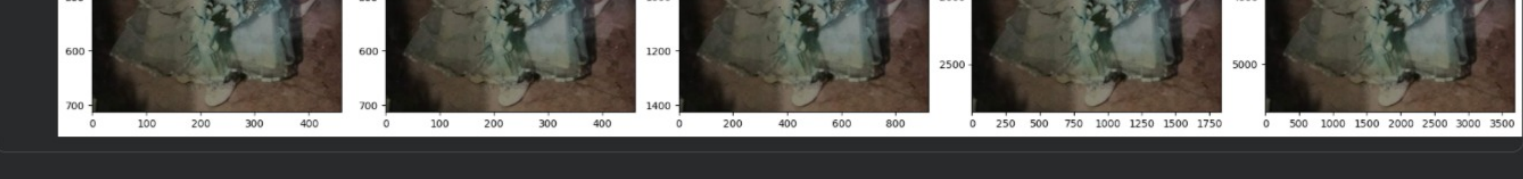
    #scale 02
    ax[2].imshow(image_2)
    ax[2].set_title('Scale 2 N=%d' % image_2.shape[0])

    #scale 04
    ax[3].imshow(image_4)
    ax[3].set_title('Scale 4 N=%d' % image_4.shape[0])

    #scale 08
    ax[4].imshow(image_8)
    ax[4].set_title('Scale 8 N=%d' % image_8.shape[0])

    plt.tight_layout()
    plt.show()

# call the function with the image file
upsample_image("Analog.png")
```



Downsampling

```
def downsample_image(Analog):
    # load the original image
    image = imread(Analog)

    # create downsampled images manually
    # resize image

    # scale 1 -> original size(no enlargement)
    image_1 = resize(
        image,
        (image.shape[0], image.shape[1]),
        anti_aliasing=True
    )

    # scale 1/2
    image_2 = resize(
        image,
        (image.shape[0]//2, image.shape[1]//2),
        anti_aliasing=True
    )

    # scale 1/4
    image_4 = resize(
        image,
        (image.shape[0]//4, image.shape[1]//4),
        anti_aliasing=True
    )

    # scale 1/8
    image_8 = resize(
        image,
        (image.shape[0]//8, image.shape[1]//8),
        anti_aliasing=True
    )

    # create plotting area (1 row and 5 columns)
    fig, ax = plt.subplots(1, 5, figsize=(20,10))

    #display image
    #original image
    ax[0].imshow(image)
    ax[0].set_title('Original N=%d' % image.shape[0])

    #scale 1
    ax[1].imshow(image_1)
    ax[1].set_title('Scale 1 N=%d' % image_1.shape[0])

    #scale 1/2
    ax[2].imshow(image_2)
    ax[2].set_title('Scale 2 N=%d' % image_2.shape[0])

    #scale 1/4
    ax[3].imshow(image_4)
    ax[3].set_title('Scale 4 N=%d' % image_4.shape[0])

    #scale 1/8
    ax[4].imshow(image_8)
    ax[4].set_title('Scale 8 N=%d' % image_8.shape[0])

    plt.tight_layout()
    plt.show()

# call the function with the image file
downsample_image("Analog.png")
```



resize() Downsampling

What it does:

- Uses interpolation (bilinear / bicubic)
- Estimates new pixel values from nearby pixels
- Works like geometric rescaling

Effect:

- Image looks smoother
- Edges are blur
- Values are interpolated (not true averaging)

🔥 It calculates new pixel values using interpolation formula

```
import matplotlib.pyplot as plt
from skimage.io import imread
from skimage.transform import downscale_local_mean

def downsample_image(Analog):
    # load the original image
    image = imread(Analog)

    # create upscaled images manually
    # resize image

    # scale 1 -> original size(no enlargement)
    image_1 = downscale_local_mean(image, (1,1,1)).astype(int)

    # scale 2
    image_2 = downscale_local_mean(image, (2,2,1)).astype(int)

    # scale 4
    image_4 = downscale_local_mean(image, (4,4,1)).astype(int)

    # scale 8
    image_8 = downscale_local_mean(image, (8,8,1)).astype(int)

    # create plotting area (1 row and 5 columns)
    fig, ax = plt.subplots(1, 5, figsize=(14,4))

    #display image
    #original image
    ax[0].imshow(image)
    ax[0].set_title('Original')

    #scale 01
    ax[1].imshow(image_1)
    ax[1].set_title('Scale 1')

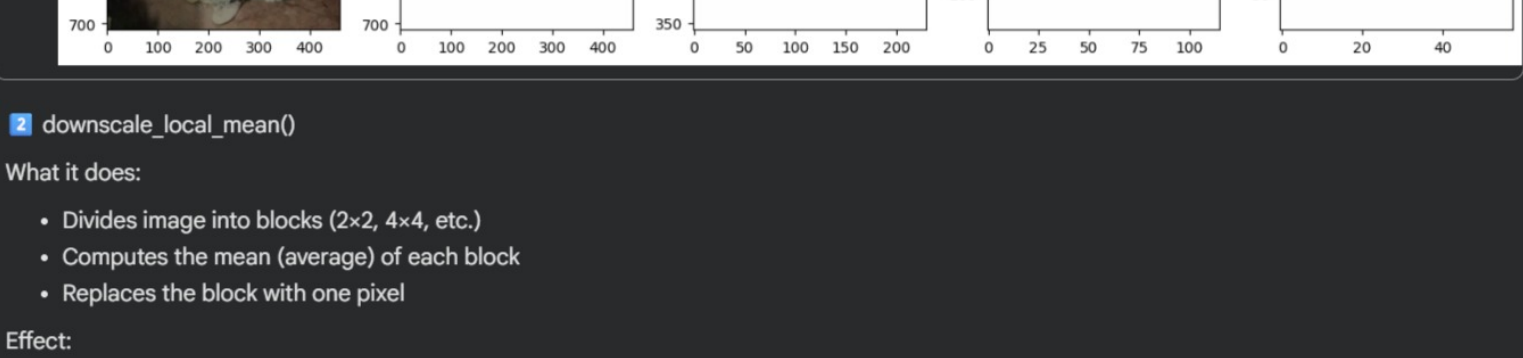
    #scale 02
    ax[2].imshow(image_2)
    ax[2].set_title('Scale 2')

    #scale 04
    ax[3].imshow(image_4)
    ax[3].set_title('Scale 4')

    #scale 08
    ax[4].imshow(image_8)
    ax[4].set_title('Scale 8')

    plt.tight_layout()
    plt.show()

# call the function with the image file
downsample_image("Analog.png")
```



downscale_local_mean()

What it does:

- Divides image into blocks (2x2, 4x4, etc.)
- Computes the mean (average) of each block
- Replaces the block with one pixel

Effect:

- Preserves overall brightness
- Reduces aliasing
- Looks slightly blurred but more mathematically correct

🔥 It performs low-pass filtering + subsampling